# Desenvolvimento Web com .NET

## Aula 7 – API

Prof. Me. Lucas Teodoro dos Santos

UNIP

30 de Setembro de 2026

# Roteiro da Aula


1. API
2. Tipos de API
3. REST e RESTful
4. Métodos HTTP
5. Códigos de Status HTTP
6. Implementação de APIs com ASP.NET Core


UNIP
UNIVERSIDADE PAULISTA

# O que é uma API?

Um software pode precisar utilizar uma funcionalidade que outro software já oferece.

Para isso, ele precisa saber **como solicitar essa funcionalidade**, **quais informações fornecer** e **qual resposta esperar**.

A **API** define essa forma de interação, permitindo que um software utilize recursos de outro sem precisar conhecer os detalhes de sua implementação.

A sigla significa *Application Programming Interface*, ou **Interface de Programação de Aplicações**.

UNIP
UNIVERSIDADE PAULISTA

# Exemplo de API

Imagine um aplicativo que apresenta a previsão do tempo de uma cidade.

Esse aplicativo pode obter as informações por meio da API de um serviço de meteorologia:

1. O aplicativo solicita a previsão, informando a cidade;
2. O serviço recebe a solicitação e consulta as informações;
3. O serviço devolve os dados da previsão;
4. O aplicativo apresenta os dados ao usuário.

A API define como realizar essa consulta. O aplicativo não precisa conhecer como o serviço coleta ou calcula a previsão do tempo.

UNIP
UNIVERSIDADE PAULISTA

# Exemplos de APIs utilizadas no mercado

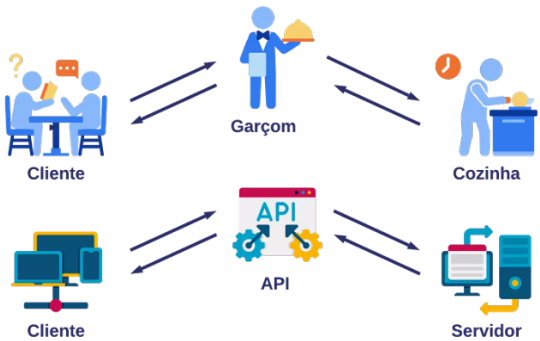
Diversas empresas disponibilizam APIs para que outros sistemas utilizem seus serviços.

- **Google Maps:** Permite incorporar mapas, pesquisar endereços e calcular rotas;
- **Mercado Pago:** Permite integrar pagamentos e consultar a situação de uma transação;
- **WhatsApp Business Platform:** Permite integrar o envio e o recebimento de mensagens aos sistemas de uma empresa;
- **Spotify:** Permite consultar informações sobre artistas, álbuns e músicas.

Essas APIs podem exigir credenciais de acesso e possuir limites de utilização ou cobrança.

UNIP
UNIVERSIDADE PAULISTA

# API: uma comparação com o garçom




Em um restaurante, o cliente faz um pedido ao garçom, que o encaminha à cozinha e traz o resultado.

- **Cliente:** Aplicação que solicita algo;
- **Garçom:** Representa a API;
- **Cozinha:** Sistema que processa o pedido;
- **Prato:** Resultado da solicitação.

Assim como o cliente não precisa conhecer o funcionamento da cozinha, a aplicação não precisa conhecer os detalhes internos do sistema.

# Tipos de API

As APIs podem ser classificadas de diferentes maneiras. Uma delas considera **como a comunicação entre as aplicações é organizada**.

Entre as principais abordagens estão:

- **REST;**
- **SOAP;**
- **GraphQL.**

Cada abordagem possui características próprias para definir as operações, estruturar as mensagens e realizar a troca de dados.

UNIP
UNIVERSIDADE PAULISTA

# API REST

REST é um **estilo arquitetural** que organiza a comunicação em torno de recursos, como produtos, usuários e pedidos.

Nas Web APIs, normalmente utiliza métodos HTTP e troca representações dos recursos em formatos como JSON.

**Vantagens:**

- Aproveita recursos do HTTP, como métodos, códigos e cache;
- Facilita a integração entre diferentes aplicações;
- Permite que cliente e servidor evoluam separadamente.

**Desvantagens e desafios:**

- Uma operação pode exigir consultas a vários endpoints;
- Respostas com estrutura fixa podem trazer dados desnecessários;
- Exige cuidado para aplicar corretamente as restrições REST.

UNIP UNIVERSIDADE PAULISTA

# API SOAP

SOAP é um **protocolo de comunicação** que utiliza mensagens estruturadas em XML.

É comum em integrações corporativas com contratos formais, frequentemente descritos em WSDL.

**Vantagens:**

- Define contratos detalhados para a comunicação;
- Possui padrões associados de segurança e confiabilidade.

**Desvantagens e desafios:**

- As mensagens XML podem ser extensas (Payload grande);
- A implementação e a configuração podem ser complexas.

UNIP
UNIVERSIDADE PAULISTA

# API GraphQL

GraphQL é uma **linguagem de consulta para APIs**, acompanhada de um modelo de execução.

O cliente informa quais campos deseja receber em cada consulta.

**Vantagens:**

- Permite solicitar apenas os dados necessários;
- Pode reunir dados relacionados em uma única consulta.

**Desvantagens e desafios:**

- A implementação e a configuração podem ser complexas.
- Consultas complexas podem sobrecarregar o servidor;
- Exige mais cuidado com permissões e limites das consultas.

UNIP
UNIVERSIDADE PAULISTA

# Qual abordagem escolher?

A escolha depende dos requisitos da aplicação e dos sistemas que irão utilizar a API.

- **REST:** adequado para integrações que buscam simplicidade na comunicação e flexibilidade no formato dos dados.
- **SOAP:** relevante quando a integração exige contratos e padrões já adotados pelo ambiente corporativo;
- **GraphQL:** útil quando diferentes clientes precisam de combinações variadas de dados;

Essas abordagens podem coexistir em uma mesma organização.

UNIP
UNIVERSIDADE PAULISTA

# API REST

# Funcionamento de uma API REST

A comunicação acontece por meio de **requisições** e **respostas HTTP**.

1. O **cliente envia uma requisição**, indicando o recurso e a operação desejada;
2. A **API recebe a requisição** e encaminha seu processamento;
3. O **servidor executa a operação**, consultando ou alterando dados quando necessário;
4. A **API devolve uma resposta**, contendo o código de status e, quando necessário, os dados;
5. O **cliente utiliza o resultado recebido**.

# Funcionamento de uma API REST

O cliente realiza uma consulta usando o método **GET** na seguinte URL:

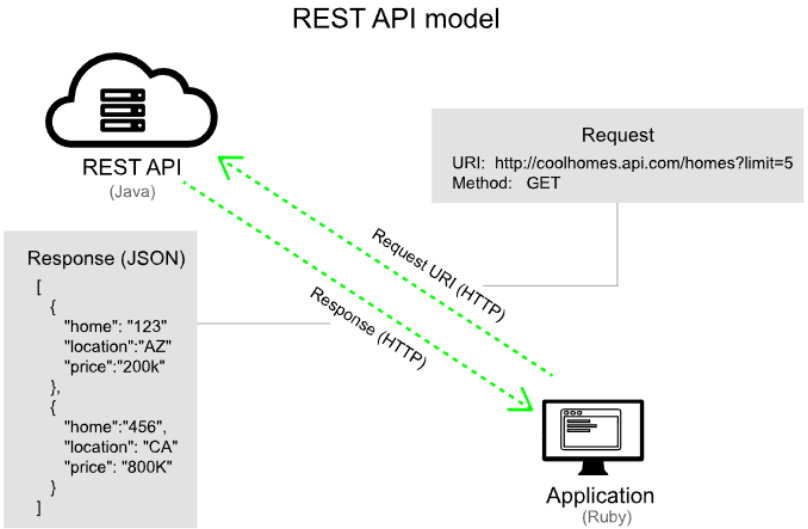
```
https://api.exemplo.com/produtos/10
```

O servidor responde com **200 OK** e os dados do produto em JSON:

```
{
    "id": 10,
    "nome": "Teclado",
    "preco": 150.00
}
```

O cliente utiliza os dados recebidos para apresentar as informações do produto ao usuário.

# Funcionamento de uma API REST



# De REST a RESTful

Entre as abordagens apresentadas, vamos aprofundar o estudo de **REST**.

REST estabelece um conjunto de princípios para organizar a comunicação entre cliente e servidor.

Quando uma API segue as restrições desse estilo, ela é denominada **RESTful**.

## Relação entre os termos

**REST** define o estilo arquitetural.
**RESTful** caracteriza uma API que segue esse estilo.

A seguir, vamos conhecer os critérios necessários para que uma API seja considerada RESTful.

UNIP
UNIVERSIDADE PAULISTA

# Quando uma API é considerada RESTful?

**REST** é um estilo arquitetural que estabelece regras para organizar a comunicação entre cliente e servidor.

Uma API é considerada **RESTful** quando segue as restrições definidas por esse estilo.

Essas restrições envolvem:

- Separação entre cliente e servidor;
- Comunicação sem estado de sessão;
- Uso de cache;
- Interface uniforme;
- Organização em camadas.

Existe também uma restrição opcional: **código sob demanda**.

UNIP
UNIVERSIDADE PAULISTA

# 1. Cliente-servidor (Client-Server)

O cliente e o servidor possuem responsabilidades diferentes.

- **Cliente:** envia solicitações e utiliza os resultados recebidos;
- **Servidor:** processa as solicitações, aplica regras e disponibiliza os recursos.

Por exemplo, um aplicativo solicita informações de um produto. O servidor fornece os dados, e o aplicativo decide como apresentá-los na tela.

Essa separação permite que cliente e servidor evoluam separadamente, preservando o contrato de comunicação.

# 2. Sem estado de sessão (Stateless)

Cada requisição deve conter as informações necessárias para que o servidor consiga compreendê-la.

O servidor não deve depender de uma conversa anterior com o cliente para entender o pedido atual.

Por exemplo, para consultar um produto, o cliente informa seu identificador e, quando necessário, as credenciais de acesso.

**Atenção**

O servidor pode armazenar dados em um banco. A restrição se refere ao estado da sessão de comunicação.

UNIP
UNIVERSIDADE PAULISTA

# 3. Uso de cache (Cacheable)

O cache permite guardar uma resposta para reutilizá-la posteriormente, respeitando suas regras de validade.

A resposta deve indicar se pode ser armazenada e reutilizada.

Por exemplo, uma lista de categorias que muda pouco pode ser reutilizada durante um período, evitando novas consultas ao servidor.

Isso pode reduzir o tempo de resposta, o tráfego de dados e a quantidade de solicitações.

**Atenção**

Nem toda resposta precisa permitir cache. A API deve comunicar as regras de reutilização.

UNIP
UNIVERSIDADE PAULISTA

# 4. Interface uniforme (Uniform Interface) 1/3

A API deve seguir uma forma consistente de identificar os recursos e organizar a comunicação.

Essa restrição envolve quatro requisitos:

- **Identificação dos recursos:** Cada recurso possui uma URI, como `/produtos/10`;
- **Representações:** O cliente manipula os recursos por meio dos dados enviados e recebidos, em formatos como JSON ou XML;
- **Mensagens autodescritivas:** Métodos, códigos de status e cabeçalhos ajudam a interpretar a comunicação;
- **Hipermídia (HATEOAS):** A resposta oferece links e controles que orientam as próximas interações disponíveis.

UNIP
UNIVERSIDADE PAULISTA

# 4. Interface uniforme (Uniform Interface) 2/3

**1. Identificação dos recursos**

A URI identifica o recurso que o cliente deseja acessar:

- `/produtos`: coleção de produtos;
- `/produtos/10`: produto com identificador 10.

**2. Manipulação por representações**

Ao consultar `/produtos/10`, o cliente pode receber uma representação em JSON:

```
{
    "id": 10,
    "nome": "Teclado",
    "preco": 150.00
}
```

Para alterar o produto, o cliente pode enviar uma representação com os novos valores.

# 4. Interface uniforme (Uniform Interface) 3/3

**3. Mensagens autodescritivas**

Na resposta de uma consulta, o código informa o resultado e o cabeçalho indica o formato dos dados:

```
HTTP/1.1 200 OK
Content-Type: application/json
```

**4. Hipermídia (HATEOAS)**

A representação pode incluir links para orientar as próximas consultas:

```
{
    "id": 10,
    "nome": "Teclado",
    "links": [
    { "rel": "self", "href": "/produtos/10" },
    { "rel": "avaliacoes", "href": "/produtos/10/avaliacoes" }
    ]
}
```

O cliente encontra na resposta o caminho para consultar as avaliações do produto.

UNIVERSIDADE PAULISTA

# 5. Organização em camadas (Layered System)

A comunicação pode passar por intermediários antes de chegar ao servidor responsável pelo recurso.

Esses intermediários podem realizar funções como:

- Distribuir requisições entre servidores;
- Armazenar respostas em cache;
- Aplicar controles de acesso.

O cliente interage com a camada que recebe sua requisição, sem precisar conhecer toda a estrutura por trás dela.

Essa restrição se refere às camadas da comunicação, como proxies e gateways.

UNIP
UNIVERSIDADE PAULISTA

# Código sob demanda (Code-on-Demand)

O servidor pode enviar código para execução no cliente, ampliando suas funcionalidades.

Um exemplo é o envio de JavaScript para execução em um navegador.

Essa é a única restrição **opcional** de REST.

Uma API pode ser RESTful sem utilizar código sob demanda.

UNIP
UNIVERSIDADE PAULISTA

# Usar HTTP e JSON torna uma API RESTful?

Utilizar HTTP, retornar JSON e oferecer operações de CRUD não é suficiente para uma API ser considerada RESTful.

Essa classificação depende do cumprimento das restrições arquiteturais apresentadas.

Na prática, muitas APIs chamadas de REST seguem apenas parte dessas restrições, especialmente quando não utilizam hipermídia.

**Ideia central**

Ser RESTful depende de como a comunicação é organizada, e não apenas das tecnologias utilizadas.

UNIP
UNIVERSIDADE PAULISTA

# Métodos HTTP

Os métodos HTTP, também chamados de **verbos HTTP**, indicam a operação que o cliente deseja realizar sobre um recurso.

Os métodos mais utilizados em APIs são:

- **GET:** Consultar recursos;
- **POST:** Enviar dados para processamento, frequentemente para Criar um recurso;
- **PUT:** Substituir os dados de um recurso;
- **PATCH:** Atualizar parcialmente um recurso;
- **DELETE:** Remover um recurso.

UNIP
UNIVERSIDADE PAULISTA

# GET: consultar recursos

O método **GET** solicita uma representação de um recurso, sem ter como objetivo alterar seus dados.

**Exemplo: consultar uma lista de produtos**

Requisição

```
GET /api/produtos
```

**Exemplo: consultar um produto específico**

Requisição

```
GET /api/produtos/1
```

A API pode responder com `200 OK` e os dados solicitados. Se o produto específico não existir, pode retornar `404 Not Found`.

# POST: criar um recurso

O método **POST** envia dados para processamento. Em um CRUD, é frequentemente utilizado para criar um recurso.

## Requisição

```
POST /api/produtos

{
 "nome": "Caderno",
 "preco": 20.00
}
```

Os dados são enviados no **corpo da requisição**.

Após criar o produto, a API pode responder com 201 `Created` e informar a URL do novo recurso no cabeçalho `Location`.

# PUT: substituir os dados de um recurso

O método **PUT** envia uma representação completa para substituir os dados de um recurso.

## Requisição

```
PUT /api/produtos/1

{
 "nome": "Caderno grande",
 "preco": 25.00
}
```

Neste exemplo, são enviados todos os campos editáveis do produto.

A API pode responder com `200 OK`, incluindo os dados atualizados, ou `204 No Content`, sem corpo de resposta.

# PATCH: atualizar parcialmente um recurso

O método **PATCH** envia alterações parciais para um recurso.

## Requisição

```
PATCH /api/produtos/1

{
 "preco": 30.00
}
```

Neste exemplo, a API aceita um JSON contendo apenas o campo que será alterado.

O preço é atualizado, enquanto o nome permanece igual. O formato das alterações deve ser definido pela API.

UNIP
UNIVERSIDADE PAULISTA

# DELETE: remover um recurso

O método **DELETE** solicita a remoção do recurso identificado pela URL.

**Requisição**

```
DELETE /api/produtos/1
```

Neste exemplo, o cliente solicita a remoção do produto com identificador `1`.

Após concluir a remoção, a API pode responder com `204 No Content`.

Esse código indica sucesso e não possui corpo de resposta.

UNIP
UNIVERSIDADE PAULISTA

# Resumo dos métodos HTTP

| **Método** | **Operação** | **Exemplo** |
|---|---|---|
| GET | Consultar lista | `/api/produtos` |
| GET | Consultar um item | `/api/produtos/1` |
| POST | Criar | `/api/produtos` |
| PUT | Substituir dados | `/api/produtos/1` |
| PATCH | Atualizar parte | `/api/produtos/1` |
| DELETE | Remover | `/api/produtos/1` |

A URL identifica o recurso e o método HTTP indica a operação.

UNIP
UNIVERSIDADE PAULISTA

# Códigos de status HTTP

O código de status HTTP informa o resultado do processamento de uma requisição.

Os códigos são organizados em cinco categorias:

- **1xx – Informações:** comunicam etapas intermediárias do processamento;
- **2xx – Sucesso:** indicam que a requisição foi atendida com sucesso;
- **3xx – Redirecionamento:** indicam redirecionamento ou condições relacionadas ao cache;
- **4xx – Erro do cliente:** indicam problemas na requisição ou no acesso ao recurso;
- **5xx – Erro do servidor:** indicam falhas do servidor ao atender a requisição.

UNIP
UNIVERSIDADE PAULISTA

# Códigos de status HTTP mais comuns

| Código | Nome | Significado |
|---|---|---|
| 200 | OK | Requisição concluída com sucesso. |
| 201 | Created | Novo recurso criado com sucesso. |
| 204 | No Content | Sucesso, sem corpo de resposta. |
| 400 | Bad Request | Requisição inválida. |
| 401 | Unauthorized | Autenticação ausente ou inválida. |
| 403 | Forbidden | Acesso ou operação não permitido. |
| 404 | Not Found | Recurso não encontrado. |
| 405 | Method Not Allowed | Método HTTP não permitido para o recurso. |
| 409 | Conflict | Conflito com o estado atual do recurso. |
| 500 | Internal Server Error | Falha inesperada no servidor. |
| 503 | Service Unavailable | Serviço temporariamente indisponível. |

# Exemplo de resposta com sucesso

## Requisição

```
GET /api/produtos/1
```

## Resposta

```
HTTP/1.1 200 OK
Content-Type: application/json

{
 "id": 1,
 "nome": "Caderno",
 "preco": 20.00
}
```

O código indica sucesso e o corpo apresenta os dados do produto.

# Exemplo de resposta com erro

## Requisição

```
GET /api/produtos/999
```

## Resposta

```
HTTP/1.1 404 Not Found
Content-Type: application/json

{
 "mensagem": "Produto nao encontrado."
}
```

O código indica que o recurso não foi encontrado. Neste exemplo, o corpo apresenta uma mensagem explicativa.

# Implementação de uma API

A implementação de uma API transforma as operações disponíveis em pontos de acesso que podem ser utilizados por outras aplicações.

Para cada operação, são definidos:

- A rota de acesso;
- O método HTTP aceito;
- Os dados recebidos;
- O processamento realizado;
- Os dados e o código de status retornados.

UNIP
UNIVERSIDADE PAULISTA

# Endpoints

Um **endpoint** é um ponto de acesso a uma operação da API, identificado pela combinação de uma rota e um método HTTP.

## Exemplos

`GET /api/produtos` — consulta os produtos.

`POST /api/produtos` — cadastra um produto.

Embora utilizem a mesma rota, os endpoints executam operações diferentes conforme o método HTTP recebido.

UNIP
UNIVERSIDADE PAULISTA

# APIs com ASP.NET Core

O **ASP.NET Core** oferece recursos para receber requisições, encaminhá-las às operações correspondentes e produzir respostas HTTP.

A implementação dos endpoints pode ser organizada de duas formas:

- **Minimal APIs:** associam rotas e métodos HTTP diretamente às funções que executam as operações;
- **Controllers:** agrupam as operações em classes, utilizando atributos para indicar as rotas e os métodos HTTP.

Ambas permitem receber dados e retornar respostas com códigos de status HTTP.

UNIP
UNIVERSIDADE PAULISTA

# Minimal APIs

Nas **Minimal APIs**, os endpoints são definidos por chamadas que associam uma rota a uma função.

- Utilizam métodos como `MapGet` e `MapPost`;
- Exigem menos código inicial;
- Podem ser definidos no `Program.cs` ou organizados em arquivos separados;
- Não exigem classes de controller.

# APIs com Controllers

Nas APIs com **Controllers**, os endpoints são organizados em métodos de classes que herdam de `ControllerBase`.

- Agrupam operações relacionadas em uma classe;
- Utilizam `[Route]` para definir as rotas;
- Utilizam atributos como `[HttpGet]` e `[HttpPost]` para indicar o método HTTP;
- Cada ação representa uma operação disponibilizada pela API.

# Diferença na definição de um endpoint na Minimal API e Controller

## Minimal API

```
app.MapGet("/api/ola", () => "Ola!");
```

## Controller

```
[ApiController]
[Route("api/ola")]
public class OlaController : ControllerBase
{
    [HttpGet]
    public IActionResult Get()
    {
        return Ok("Ola!");
    }
}
```

Ambos atendem a `GET /api/ola` e retornam `200 OK` com o texto `Olá!`.

UNIP
UNIVERSIDADE PAULISTA

# Comparação entre as abordagens

| **Aspecto** | **Minimal APIs** | **Controllers** |
|---|---|---|
| Definição | Métodos como MapGet e MapPost | Atributos como [HttpGet] e [HttpPost] |
| Organização | Funções e grupos de endpoints | Classes e métodos de ação |
| Código inicial | Mais enxuto | Mais estruturado em classes |
| Rotas | Informadas no mapeamento | Definidas por atributos |

As duas abordagens permitem implementar APIs completas. A escolha depende da organização e dos recursos desejados.

# Resumo geral da aula

- Conceito de API e princípios da arquitetura REST;
- Métodos HTTP e códigos de status das respostas;
- Implementação de uma API;
- Endpoints;
- Diferenças entre Minimal APIs e Controllers;

UNIP
UNIVERSIDADE PAULISTA

# Obrigado!

Dúvidas?

UNIP
UNIVERSIDADE PAULISTA